

实验七：文件系统实验报告

[仓库地址在此处](#)

本阶段代码请通过 `git checkout fs-xv6` 来查看。

一、实验基本信息

（一）实验目的

通过深入分析 xv6 的简化文件系统，理解现代文件系统的核心概念和实现原理，独立实现一个功能完整的文件系统。重点掌握磁盘布局、inode 管理、块缓存、日志系统、目录操作等关键技术。

（二）实验环境

- 操作系统：xv6 (RISC-V 版本)
- 开发环境：QEMU 模拟器、GCC 工具链
- 磁盘设备：virtio-blk 模拟磁盘

二、实验核心原理分析

（一）xv6 文件系统整体架构

xv6 文件系统采用经典的 UNIX 文件系统设计，包含以下核心组件：

- 磁盘布局管理：将磁盘划分为引导区、超级块、日志区、inode 区、位图区和数据区
- inode 机制：文件元数据管理，支持直接和间接块映射
- 块缓存系统：减少磁盘 I/O，提高读写性能
- 日志系统：保证文件系统操作的原子性和一致性
- 目录结构：简单的目录项组织方式

（二）关键数据结构分析

1. 超级块结构

```
1 struct superblock {  
2     uint magic;           // 文件系统魔数，用于识别文件系统类型  
3     uint size;           // 文件系统总块数  
4     uint nblocks;        // 数据块数量  
5     uint ninodes;        // inode 数量  
6     uint nlog;           // 日志块数量  
7     uint logstart;       // 日志区起始块号  
8     uint inodestart;     // inode 区起始块号  
9     uint bmpstart;       // 位图起始块号  
10 };
```

2. 磁盘 inode 结构

```
1 struct dinode {  
2     short type;          // 文件类型（普通文件、目录、设备文件）  
3     short major;         // 主设备号
```

```

4     short minor;          // 次设备号
5     short nlink;         // 硬链接计数
6     uint size;           // 文件大小（字节）
7     uint addrs[NDIRECT+1]; // 数据块指针数组
8 };

```

3. 内存 inode 结构

```

1 struct inode {
2     uint dev;          // 设备号
3     uint inum;         // inode号
4     int ref;           // 引用计数
5     struct sleeplock lock; // 保护锁
6     int valid;         // 内容是否有效
7
8     // 磁盘inode的拷贝
9     short type;
10    short major;
11    short minor;
12    short nlink;
13    uint size;
14    uint addrs[NDIRECT+1];
15 };

```

三、实验任务详细实现

（一）任务 1：理解 xv6 文件系统布局

1. 磁盘布局结构分析

| boot | super | log | inode blocks | bitmap | data blocks | | 0 | 1 | 2-? | ?-? | ? | ?-end |

各区域作用分析：

- 引导区（boot）：存储引导程序，xv6 中通常为空白
- 超级块（super）：存储文件系统元数据，用于识别和挂载文件系统
- 日志区（log）：实现事务日志，保证操作原子性
- inode 区：存储所有文件的元数据（inode）
- 位图区（bitmap）：跟踪数据块的使用情况
- 数据区：存储实际文件内容

布局设计原理：

- 顺序访问优化：相关数据块就近存放，减少磁头寻道时间
- 元数据集中：超级块和 inode 表放在固定位置，便于快速访问
- 日志隔离：日志区独立，避免日志操作影响正常数据

区域大小确定因素：

- 超级块：固定大小（1 个块）
- 日志区：根据并发事务数量确定，通常为 30-100 个块

- inode 区：根据预计文件数量计算（每个 inode 64 字节）
- 位图区：根据数据块数量计算（1 位对应 1 个块）
- 数据区：剩余空间

2. 超级块的关键作用 为什么需要这些元数据：

- magic：验证文件系统类型，防止误挂载
- size：确定文件系统边界，避免越界访问
- nblocks/ninodes：资源管理的基础
- 区域起始位置：快速定位各个区域

一致性保证机制：

- 写前日志：所有元数据修改先写入日志
- 冗余备份：可考虑多副本超级块
- 校验和：检测元数据损坏

3. inode 结构深入分析 直接块和间接块设计：

- 直接块：12 个直接指针，支持小文件快速访问
- 间接块：1 个一级间接指针，支持中等文件
- 最大文件大小计算： $12 * BSIZE + (BSIZE / \text{sizeof}(\text{uint})) * BSIZE$

硬链接实现：

- 多个目录项指向同一 inode
- nlink 计数跟踪引用数
- 只有 nlink=0 且无进程打开时才真正删除

深入思考解答：

1. 简单布局的优缺点：

- 优点：实现简单、可靠性高、易于调试
- 缺点：扩展性差、空间利用率低、性能有限

2. 提高空间利用率：

- 块分配策略优化（如 extent 分配）
- 小文件内联存储
- 压缩技术应用

3. 现代文件系统改进：

- B+树索引结构（如 ext4、XFS）
- 日志结构文件系统（LFS）
- Copy-on-Write 技术（ZFS、Btrfs）
- 快照和去重功能

（二）任务 2：分析 xv6 的 inode 管理机制

1. inode 缓存管理实现

```
1 // inode缓存查找函数
2 struct inode* iget(uint dev, uint inum) {
```



```
3     struct inode *ip, *empty;
4
5     acquire(&icache.lock);
6
7     // 查找缓存中是否已存在
8     empty = 0;
9     for(ip = &icache.inode[0]; ip < &icache.inode[NINODE]; ip++) {
10         if(ip->ref > 0 && ip->dev == dev && ip->inum == inum) {
11             ip->ref++;
12             release(&icache.lock);
13             return ip;
14         }
15         if(empty == 0 && ip->ref == 0) // 记录空槽位
16             empty = ip;
17     }
18
19     // 缓存未命中，分配新项
20     if(empty == 0)
21         panic("iget: no inodes");
22
23     ip = empty;
24     ip->dev = dev;
25     ip->inum = inum;
26     ip->ref = 1;
27     ip->valid = 0;
28     release(&icache.lock);
29
30     return ip;
31 }
```

缓存一致性保证：

- 读写锁保护：多个读可并发，写操作独占
- 有效位标记：避免重复从磁盘读取
- 引用计数：确保正在使用的 inode 不被回收

2. inode 分配算法分析

```
1 struct inode* ialloc(uint dev, short type) {
2     struct superblock sb;
3     struct buf *bp;
4     struct dinode *dip;
5
6     // 读取超级块获取inode区域信息
7     readsb(dev, &sb);
8
9     for(uint inum = 1; inum < sb.ninodes; inum++) {
10         // 计算inode所在的块和偏移
```

```

11     uint block = IBLOCK(inum, sb);
12     bp = bread(dev, block);
13     dip = (struct dinode*)bp->data + (inum % IPB);
14
15     if(dip->type == 0) { // 空闲inode
16         memset(dip, 0, sizeof(*dip));
17         dip->type = type;
18         log_write(bp); // 写入日志
19         brelse(bp);
20         return iget(dev, inum);
21     }
22     brelse(bp);
23 }
24 panic("ialloc: no inodes");
25 }

```

并发分配同步：

- 位图操作原子性：通过日志系统保证
- 缓存锁保护：防止重复分配同一 inode

3. 数据块映射机制

```

1  static uint bmap(struct inode *ip, uint bn) {
2      uint addr, *a;
3      struct buf *bp;
4
5      // 直接块处理
6      if(bn < NDIRECT) {
7          if((addr = ip->addrs[bn]) == 0) {
8              addr = balloc(ip->dev);
9              if(addr == 0) return 0;
10             ip->addrs[bn] = addr;
11             iupdate(ip); // 更新磁盘inode
12         }
13         return addr;
14     }
15
16     // 间接块处理
17     bn -= NDIRECT;
18     if(bn < NINDIRECT) {
19         if((addr = ip->addrs[NDIRECT]) == 0) {
20             addr = balloc(ip->dev);
21             if(addr == 0) return 0;
22             ip->addrs[NDIRECT] = addr;
23             iupdate(ip);
24         }
25         bp = bread(ip->dev, addr);
26         a = (uint*)bp->data;

```

```

27         if((addr = a[bn]) == 0) {
28             addr = balloc(ip->dev);
29             if(addr) {
30                 a[bn] = addr;
31                 log_write(bp);
32             }
33         }
34         brelse(bp);
35         return addr;
36     }
37
38     panic("bmap: out of range");
39 }

```

关键问题解答：

1. inode 缓存替换策略：简单的全局 LRU 链表，当缓存满时淘汰引用计数为 0 的 inode
2. 防止 inode 泄漏：
 - 引用计数精确管理
 - 文件关闭时确保 iput 调用
 - 进程退出时清理所有打开文件
3. 大文件性能优化：
 - 多级间接块支持更大文件
 - 预读技术提高顺序访问性能
 - 块分配策略优化减少碎片

（三）任务 3：设计改进的文件系统布局

改进的文件系统设计

```

1  #define BLOCK_SIZE 4096           // 增大块大小，提高大文件性能
2  #define SUPERBLOCK_NUM 1         // 超级块位置
3  #define SUPERBLOCK_BACKUP 1024   // 备份超级块
4  #define LOG_START 2              // 日志区起始
5  #define LOG_SIZE 100             // 更大的日志区
6  #define INODE_BLOCKS 256         // 更多的inode空间
7
8  // 改进的inode结构
9  struct my_inode {
10     uint16_t mode;                // 文件模式和类型
11     uint16_t uid;                 // 所有者ID
12     uint16_t gid;                 // 组ID
13     uint32_t size;                // 文件大小
14     uint32_t blocks;              // 分配的块数
15     uint32_t atime;               // 访问时间
16     uint32_t mtime;              // 修改时间
17     uint32_t ctime;               // 创建时间

```

```

18     uint32_t direct[12];        // 直接块指针
19     uint32_t indirect;         // 一级间接块
20     uint32_t double_indirect;  // 二级间接块
21     uint32_t flags;            // 扩展标志位
22 };

```

设计权衡考虑：

1. 小文件和大文件效率平衡：
 - 直接块优化小文件（12 个直接块）
 - 多级间接块支持大文件
 - 内联数据：极小文件直接存储在 inode 中
2. 扩展属性支持：
 - 预留 flag 字段用于功能标记
 - 可扩展的元数据区域
3. 目录性能优化：
 - B 树索引替代线性目录
 - 目录项哈希加速查找
4. 符号链接支持：
 - 特殊文件类型标记
 - 短链接内联存储，长链接使用数据块

（四）任务 4：实现块缓存系统

改进的块缓存设计

```

1  #define NBUF 1000                // 更大的缓存池
2  #define HASH_SIZE 101           // 哈希表大小
3
4  struct buffer_head {
5      uint32_t block_num;         // 块号
6      int dev;                   // 设备号
7      char *data;                // 数据指针
8      int dirty;                 // 脏位
9      int valid;                 // 有效位
10     int ref_count;              // 引用计数
11     struct spinlock lock;       // 缓存项锁
12
13     // 哈希链表
14     struct buffer_head *hash_next;
15     struct buffer_head *hash_prev;
16
17     // LRU链表
18     struct buffer_head *lru_next;
19     struct buffer_head *lru_prev;
20 };
21

```

```
22 struct bcache {
23     struct spinlock lock;
24     struct buffer_head buf[NBUF];
25     struct buffer_head lru_head; // LRU链表头
26     struct buffer_head *hash_table[HASH_SIZE];
27 };
```

核心缓存函数实现

```
1 // 块读取函数 
2 struct buffer_head* get_block(uint dev, uint block) {
3     struct buffer_head *bh;
4     uint hash = block % HASH_SIZE;
5
6     acquire(&bcache.lock);
7
8     // 查找哈希表
9     for(bh = bcache.hash_table[hash]; bh; bh = bh->hash_next) {
10         if(bh->dev == dev && bh->block_num == block) {
11             bh->ref_count++;
12             // 从LRU链表移除（最近使用）
13             remove_from_lru(bh);
14             release(&bcache.lock);
15             return bh;
16         }
17     }
18
19     // 缓存未命中，分配新项
20     bh = find_lru_buffer();
21     if(bh->dirty) {
22         // 写回脏数据
23         write_block(bh);
24     }
25
26     // 设置新块信息
27     bh->dev = dev;
28     bh->block_num = block;
29     bh->ref_count = 1;
30     bh->dirty = 0;
31
32     // 从磁盘读取数据
33     read_disk_block(dev, block, bh->data);
34     bh->valid = 1;
35
36     // 插入哈希表
37     insert_into_hash(bh, hash);
38     release(&bcache.lock);
39 }
```



```

40     return bh;
41 }
42
43 // 块释放函数
44 void put_block(struct buffer_head *bh) {
45     acquire(&bcache.lock);
46     bh->ref_count--;
47     if(bh->ref_count == 0) {
48         // 插入LRU链表尾部
49         insert_into_lru(bh);
50     }
51     release(&bcache.lock);
52 }
53
54 // 同步块数据
55 void sync_block(struct buffer_head *bh) {
56     if(bh->dirty) {
57         write_disk_block(bh->dev, bh->block_num, bh->data);
58         bh->dirty = 0;
59     }
60 }
61
62 // 刷新所有块
63 void flush_all_blocks(uint dev) {
64     struct buffer_head *bh;
65     acquire(&bcache.lock);
66     for(int i = 0; i < NBUF; i++) {
67         bh = &bcache.buf[i];
68         if(bh->dev == dev && bh->dirty) {
69             sync_block(bh);
70         }
71     }
72     release(&bcache.lock);
73 }

```

缓存策略考虑：

1. 缓存大小确定：根据可用内存和工作集大小动态调整
2. 写回时机：定期同步、缓存满时、文件系统卸载时
3. I/O 错误处理：重试机制、坏块标记、错误报告
4. 预读策略：顺序访问模式检测，提前读取相邻块

（五）任务 5：实现日志系统

日志系统状态管理

```

1  struct log_header {
2      int n;                // 日志中的操作数
3      int block[LOGSIZE];   // 操作的块号

```



```
4  };
5
6  struct log {
7      struct spinlock lock;
8      int start;           // 日志区起始块
9      int size;            // 日志区大小
10     int outstanding;      // 未完成的事务数
11     int committing;       // 是否正在提交
12     int dev;              // 设备号
13     struct log_header lh; // 日志头
14 };
```

事务处理核心函数

```
1  // 日志初始化
2  void log_init(int dev, struct superblock *sb) {
3      log.dev = dev;
4      log.start = sb->logstart;
5      log.size = sb->nlog;
6      initlock(&log.lock, "log");
7      recover_from_log(); // 崩溃恢复
8  }
9
10 // 开始事务
11 void begin_transaction(void) {
12     acquire(&log.lock);
13     log.outstanding++;
14     release(&log.lock);
15 }
16
17 // 记录写操作
18 void log_write(struct buffer_head *bh) {
19     acquire(&log.lock);
20
21     if(log.lh.n >= LOGSIZE) {
22         panic("log_write: too many blocks in transaction");
23     }
24
25     // 检查是否已记录该块
26     for(int i = 0; i < log.lh.n; i++) {
27         if(log.lh.block[i] == bh->block_num) {
28             // 已记录, 直接返回
29             release(&log.lock);
30             return;
31         }
32     }
33
34     // 记录新块
```

```
35     log.lh.block[log.lh.n] = bh->block_num;
36     log.lh.n++;
37
38     // 将数据拷贝到日志块
39     struct buffer_head *lbh = get_log_block(log.start + log.lh.n);
40     memmove(lbh->data, bh->data, BLOCK_SIZE);
41     lbh->dirty = 1;
42     put_block(lbh);
43
44     release(&log.lock);
45 }
46
47 // 提交事务
48 void commit_transaction(void) {
49     acquire(&log.lock);
50
51     if(log.committing) {
52         panic("commit: already committing");
53     }
54
55     log.committing = 1;
56
57     // 将日志头写入磁盘
58     write_log_header();
59
60     // 执行提交: 将日志数据写入目标位置
61     install_trans();
62
63     // 清空日志头
64     log.lh.n = 0;
65     write_log_header();
66
67     log.committing = 0;
68     log.outstanding--;
69
70     // 唤醒等待的进程
71     wakeup(&log);
72
73     release(&log.lock);
74 }
75
76 // 崩溃恢复
77 void recover_from_log(void) {
78     read_log_header();
79
80     if(log.lh.n > 0) {
81         // 有未完成的事务, 重新执行
```

```

82     install_trans();
83     log.lh.n = 0;
84     write_log_header();
85 }
86 }

```

日志系统设计要点：

1. 日志大小确定：根据最大并发事务数和平均事务大小计算
2. 日志满处理：限制单事务大小、强制提交、等待空间释放
3. 原子性保证：先写日志后写数据，恢复时重做或撤销
4. 性能优化：组提交、日志压缩、异步提交

(六) 任务 6：实现目录和路径解析

目录操作接口

```

1  // 目录项查找
2  struct inode* dir_lookup(struct inode *dp, char *name, uint *poff) {
3      if(dp->type != T_DIR) {
4          panic("dir_lookup: not a directory");
5      }
6
7      struct dirent de;
8      for(uint off = 0; off < dp->size; off += sizeof(de)) {
9          if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de)) {
10             panic("dir_lookup: read error");
11         }
12         if(de.inum == 0) {
13             continue; // 空闲目录项
14         }
15         if(strcmp(name, de.name) == 0) {
16             // 找到匹配项
17             if(poff) *poff = off;
18             return iget(dp->dev, de.inum);
19         }
20     }
21
22     return 0; // 未找到
23 }
24
25 // 创建目录项
26 int dir_link(struct inode *dp, char *name, uint inum) {
27     // 检查名称是否已存在
28     if(dir_lookup(dp, name, 0) != 0) {
29         return -1; // 已存在
30     }
31
32     // 查找空闲目录项

```

```
33     struct dirent de;
34     for(uint off = 0; off < dp->size; off += sizeof(de)) {
35         if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de)) {
36             return -1;
37         }
38         if(de.inum == 0) {
39             // 找到空闲项
40             strncpy(de.name, name, DIRSIZ);
41             de.inum = inum;
42             if(writei(dp, (char*)&de, off, sizeof(de)) != sizeof(de)) {
43                 return -1;
44             }
45             return 0;
46         }
47     }
48
49     // 需要扩展目录
50     uint off = dp->size;
51     de.inum = inum;
52     strncpy(de.name, name, DIRSIZ);
53     dp->size += sizeof(de);
54     iupdate(dp);
55
56     if(writei(dp, (char*)&de, off, sizeof(de)) != sizeof(de)) {
57         return -1;
58     }
59
60     return 0;
61 }
62
63 // 路径解析
64 struct inode* path_walk(char *path) {
65     struct inode *ip, *next;
66     char name[DIRSIZ];
67
68     if(*path == '/') {
69         ip = iget_root(); // 根目录
70         path++;
71     } else {
72         ip = idup(myproc()->cwd); // 当前目录
73     }
74
75     while(*path != '\0') {
76         // 跳过斜杠
77         while(*path == '/') path++;
78         if(*path == '\0') break;
79     }
```

```
80      // 获取路径组件
81      get_path_component(path, name);
82      path += strlen(name);
83
84      ilock(ip);
85      if(ip->type != T_DIR) {
86          iunlock(ip);
87          iput(ip);
88          return 0;
89      }
90
91      next = dir_lookup(ip, name, 0);
92      iunlock(ip);
93      iput(ip);
94
95      if(next == 0) {
96          return 0; // 未找到
97      }
98      ip = next;
99  }
100
101  return ip;
102 }
```

目录设计考虑：

1. 目录大小限制：动态扩展，受文件系统大小限制
2. 长文件名支持：可变长度目录项设计
3. 遍历效率优化：B 树索引、目录项缓存
4. 链接处理：硬链接直接指向 inode，符号链接特殊处理

四、测试与调试策略

（一）文件系统完整性测试

```
1  void test_filesystem_integrity(void) {
2      printf("Testing filesystem integrity...\n");
3
4      // 创建测试文件
5      int fd = open("testfile", O_CREATE | O_RDWR);
6      assert(fd >= 0);
7
8      // 写入数据
9      char buffer[] = "Hello, filesystem!";
10     int bytes = write(fd, buffer, strlen(buffer));
11     assert(bytes == strlen(buffer));
12
13     // 同步到磁盘
```

```
14     fsync(fd);
15     close(fd);
16
17     // 重新打开并验证
18     fd = open("testfile", O_RDONLY);
19     assert(fd >= 0);
20
21     char read_buffer[64];
22     bytes = read(fd, read_buffer, sizeof(read_buffer));
23     read_buffer[bytes] = '\0';
24     assert(strcmp(buffer, read_buffer) == 0);
25
26     close(fd);
27
28     // 删除文件
29     assert(unlink("testfile") == 0);
30
31     printf("Filesystem integrity test passed\n");
32 }
```

本实验还拓展实现了 `shell` 功能。测试下面的 `initcode.c` 代码来测试文件读写：

```
1  #include "user.h"
2  #include "memlayout.h"
3  #include "../src/fcntl.h"
4
5  void test_fork();
6
7  void test_sbrk();
8
9  void test_uptime();
10
11 void test_gettimeofday();
12
13 void test_read();
14
15 void test_shell();
16
17 int main()
18 {
19     if (open("console", O_RDWR) < 0)
20     {
21         mknod("console", 1, 1);
22         open("console", O_RDWR);
23     }
24     dup(0); // stdout
25     dup(0); // stderr
26 }
```

```
27     // test_sbrk();
28
29     // test_fork();
30
31     // test_uptime();
32
33     // test_gettimeofday();
34
35     test_read();
36
37     test_shell();
38
39     shutdown();
40     return 0;
41 }
42
43 void test_fork()
44 {
45
46     printfYellow("===== Test Fork =====\n");
47     int pid = fork();
48     int status;
49     if (pid < 0)
50     {
51         printf("Fork failed!\n");
52     }
53     else if (pid == 0)
54     {
55         // Child process
56         printf("Hello from child process! PID: %d\n", getpid());
57         exit(0);
58     }
59     else
60     {
61         // Parent process
62         wait(&status);
63         printf("Hello from parent process! PID: %d, Child PID: %d\n",
64             getpid(), pid);
65         printf("Child exited with status: %d\n", status);
66     }
67 }
68 void test_sbrk()
69 {
70
71     printfYellow("===== Test sbrk =====\n");
72
```



```
73     long long addr = sbrk(0);
74
75     printf("Current break address: %p\n", addr);
76
77     sbrk(addr + 4096); // 增加4KB
78
79     // 测试是否成功增加
80     // 写入 'test sbrk'
81
82     char *test_str = "test sbrk";
83     int n = 9;
84     for (int i = 0; i < n; i++)
85     {
86         *((char *)addr + i) = test_str[i];
87     }
88     *((char *)addr + n) = '\0';
89     printf("%s\n", (char *)addr);
90 }
91
92 void test_uptime()
93 {
94     printfYellow("===== Test Uptime =====\n");
95
96     uint64 start = uptime();
97     printf("System has been up for %d ticks\n", start);
98
99     // 睡眠 1 秒 (10个滴答)
100    sleep(10);
101
102    uint64 end = uptime();
103    printf("After sleep: %d ticks elapsed\n", end - start);
104 }
105
106 void test_gettimeofday()
107 {
108     printfYellow("===== Test GetTimeOfDay =====\n");
109
110     struct timeval start, end;
111
112     gettimeofday(&start);
113     printf("Start: %d seconds, %d microseconds\n", start.tv_sec,
114           start.tv_usec);
115
116     // 做一些工作
117     sleep(20); // 睡眠 2 秒
118
119     gettimeofday(&end);
```

```
119     printf("End: %d seconds, %d microseconds\n", end.tv_sec,
120           end.tv_usec);
121
122     uint64 elapsed = (end.tv_sec - start.tv_sec) * 1000000 +
123                     (end.tv_usec - start.tv_usec);
124     printf("Elapsed: %d microseconds\n", elapsed);
125 }
126
127 void test_read()
128 {
129     printfYellow("===== Test Read README =====\n");
130
131     int fd;
132     char buffer[512];
133     int n;
134
135     // 打开 README 文件
136     fd = open("README", O_RDONLY);
137     if (fd < 0)
138     {
139         printfYellow("Failed to open README file\n");
140         return;
141     }
142     printf("Successfully opened README file, fd: %d\n", fd);
143
144     // 读取文件内容
145     printf("File content:\n");
146     printfYellow("-----\n");
147
148     while ((n = read(fd, buffer, sizeof(buffer) - 1)) > 0)
149     {
150         buffer[n] = '\0'; // 添加字符串结束符
151         printf("%s", buffer);
152     }
153
154     printfYellow("-----\n");
155
156     if (n < 0)
157     {
158         printfYellow("Error reading file\n");
159     }
160
161     close(fd);
162     printf("File closed successfully\n");
163 }
164 void test_shell()
```

```

165 {
166     char *argv[] = {"sh", 0};
167     printfYellow("===== Test Shell =====\n");
168     int pid, wpid, status;
169     for (;;)
170     {
171         printf("init: starting sh\n");
172         pid = fork();
173         if (pid < 0)
174         {
175             printf("init: fork failed\n");
176             exit(-1);
177         }
178         if (pid == 0)
179         {
180             exec("sh", argv);
181             printf("init: exec sh failed\n");
182             exit(-1);
183         }
184         while ((wpid = wait(&status)) >= 0 && wpid != pid)
185         {
186             // printf("zombie!\n");
187         }
188     }
189 }

```

运行结果如下图所示：

```

[INFO] xv6 is booting!
[INFO] virtio disk init 0
===== Test Read README =====
Successfully opened README file, fd: 3
File content:
-----
这里是 README
-----
File closed successfully
===== Test Shell =====
init: starting sh
$

```

读取的内容与 README 实际内容一致。

```

You, 6天前 | 1 author (You)
1 这里是 README You, 6天前 • 测试 sys_read 系统调用 ...
2

```

故功能完全正确。

(二) 并发访问测试

```
1 void test_concurrent_access(void) {
2     printf("Testing concurrent file access\n");
3
4     // 创建多个进程同时访问文件系统
5     for(int i = 0; i < 4; i++) {
6         if(fork() == 0) {
7             // 子进程: 创建和删除文件
8             char filename[32];
9             snprintf(filename, sizeof(filename), "test_%d", i);
10
11             for(int j = 0; j < 100; j++) {
12                 int fd = open(filename, O_CREATE | O_RDWR);
13                 if(fd >= 0) {
14                     write(fd, &j, sizeof(j));
15                     close(fd);
16                 }
17                 unlink(filename);
18             }
19             exit(0);
20         }
21     }
22
23     // 等待所有子进程完成
24     for(int i = 0; i < 4; i++) {
25         wait(NULL);
26     }
27
28     printf("Concurrent access test completed\n");
29 }
```

(三) 崩溃恢复测试

```
1 void test_crash_recovery(void) {
2     printf("Testing crash recovery...\n");
3
4     // 开始事务
5     begin_transaction();
6
7     // 执行关键文件操作
8     int fd = open("critical_file", O_CREATE | O_RDWR);
9     write(fd, "important data", 14);
10
11     // 模拟崩溃: 在提交前重启
12     // 实际测试中需要特殊框架支持
13     simulate_crash_before_commit();
14 }
```

```
15 // 重启后检查恢复情况
16 recover_from_log();
17
18 // 验证文件系统一致性
19 check_filesystem_consistency();
20
21 printf("Crash recovery test completed\n");
22 }
```

(四) 性能测试与优化

```
1 void test_filesystem_performance(void) {
2     printf("Testing filesystem performance...\n");
3
4     uint64 start_time = get_time();
5
6     // 大量小文件测试
7     for(int i = 0; i < 1000; i++) {
8         char filename[32];
9         snprintf(filename, sizeof(filename), "small_%d", i);
10        int fd = open(filename, O_CREATE | O_RDWR);
11        write(fd, "test", 4);
12        close(fd);
13    }
14    uint64 small_files_time = get_time() - start_time;
15
16    // 大文件测试
17    start_time = get_time();
18    int fd = open("large_file", O_CREATE | O_RDWR);
19    char large_buffer[4096];
20    for(int i = 0; i < 1024; i++) { // 4MB文件
21        write(fd, large_buffer, sizeof(large_buffer));
22    }
23    close(fd);
24    uint64 large_file_time = get_time() - start_time;
25
26    printf("Small files (1000x4B): %lu cycles\n", small_files_time);
27    printf("Large file (1x4MB): %lu cycles\n", large_file_time);
28
29    // 清理测试文件
30    for(int i = 0; i < 1000; i++) {
31        char filename[32];
32        snprintf(filename, sizeof(filename), "small_%d", i);
33        unlink(filename);
34    }
35    unlink("large_file");
36 }
```

五、思考题深入分析

（一）设计权衡问题

1. xv6 简单文件系统有什么优缺点
2. 简单性与性能的平衡：
 - xv6 的简单设计优势：
 - 代码可读性强，便于教学和理解
 - 调试和维护成本低
 - 可靠性高，边界情况少
 - 性能瓶颈：
 - 线性目录查找： $O(n)$ 时间复杂度
 - 固定大小块分配：可能产生内部碎片
 - 简单的 LRU 缓存：缺乏智能预读
 - 平衡策略：
 - 核心机制保持简单，关键路径优化
 - 渐进式改进，避免过度工程
 - 针对工作负载特性优化

（二）一致性保证机制

1. 日志系统的原子性保证：

```
1 // 原子性保证的核心逻辑
2 void atomic_operation(void) {
3     begin_op();
4
5     // 所有修改先写入日志
6     for each modified block {
7         log_write(bh);
8     }
9
10    // 原子提交：先写日志头，再写数据
11    commit_op();
12
13    // 恢复时：如果日志头有效，重做所有操作
14    if (log.lh.n > 0) {
15        install_trans(); // 重做操作
16        log.lh.n = 0;    // 清空日志
17        write_head();    // 确认完成
18    }
19 }
```

2. 恢复过程中的二次崩溃处理：
 - 设计原则：恢复操作必须是幂等的
 - 实现机制：
 - 恢复前检查日志完整性

- 使用序列号标识不同恢复周期
- 关键元数据更新采用谨慎替换策略
- 安全措施：
 - 恢复期间禁止新事务
 - 恢复完成前标记文件系统为“只读”
 - 提供 fsck 工具手动修复

(三) 性能优化策略

1. 主要性能瓶颈分析：

- 磁盘 I/O：机械磁盘寻道时间、固态硬盘写放大
- 锁竞争：全局 inode 缓存锁、目录操作锁
- 内存拷贝：用户空间与内核空间数据传递

2. 目录查找效率改进：

```

1  // B树目录索引实现
2  struct btree_node {
3      int is_leaf;
4      int key_count;
5      char *keys[BTREE_ORDER];
6      uint inums[BTREE_ORDER];
7      struct btree_node *children[BTREE_ORDER+1];
8  };
9
10 // 改进的目录查找
11 struct inode* btree_dir_lookup(struct inode *dp, char *name) {
12     struct btree_root *root = get_btree_root(dp);
13     return btree_search(root, name);
14 }
15
16 // 哈希目录加速
17 struct hash_dir {
18     struct spinlock lock;
19     struct hash_bucket buckets[HASH_SIZE];
20 };
21
22 struct inode* hash_dir_lookup(struct inode *dp, char *name) {
23     uint hash = hash_string(name) % HASH_SIZE;
24     acquire(&dp->hash_table.lock);
25
26     // 在哈希桶中查找
27     for each entry in dp->hash_table.buckets[hash] {
28         if (strcmp(entry.name, name) == 0) {
29             release(&dp->hash_table.lock);
30             return iget(dp->dev, entry.inum);
31         }
32     }

```

```

33
34     release(&dp->hash_table.lock);
35     return 0;
36 }

```

(四) 可扩展性设计

1. 大文件和大文件系统支持:

```

1  // 扩展的inode结构支持更大文件
2  struct ext_inode {
3      uint32_t direct[12];      // 直接块
4      uint32_t indirect;        // 一级间接 (1024块)
5      uint32_t double_indirect; // 二级间接 (1M块)
6      uint32_t triple_indirect; // 三级间接 (1G块)
7  };
8
9  // 最大文件大小计算
10 #define MAX_FILE_SIZE ( \
11     12 * BSIZE + \                // 直接块
12     (BSIZE/sizeof(uint)) * BSIZE + \ // 一级间接
13     (BSIZE/sizeof(uint)) * (BSIZE/sizeof(uint)) * BSIZE // 二级间接
14 )
15
16 // 动态inode分配
17 uint alloc_inode_dynamic(void) {
18     // 传统位图方式有限, 改为B树或位图层次结构
19     return btree_alloc_inode();
20 }

```

2. 现代文件系统特性集成:

• 写时复制 (Copy-on-Write):

```

1  // CoW数据块分配
2  uint cow_balloc(uint old_block) {
3      uint new_block = balloc();
4      // 拷贝旧数据到新块
5      copy_block(old_block, new_block);
6      return new_block;
7  }

```

• 快照功能:

```

1  // 创建快照
2  int create_snapshot(void) {
3      // 冻结文件系统状态
4      freeze_filesystem();
5      // 拷贝关键元数据
6      copy_metadata();

```



```

7    // 创建快照inode
8    return make_snapshot_inode();
9 }

```

- 数据去重:

```

1  // 块级去重
2  uint dedup_alloc(uint hash, uint size) {
3      // 查找相同内容的已有块
4      uint existing = find_duplicate_block(hash, size);
5      if (existing) {
6          // 增加引用计数
7          inc_refcount(existing);
8          return existing;
9      }
10     // 分配新块
11     return balloc();
12 }

```

(五) 可靠性保障机制

1. 文件系统损坏检测:

```

1  // 文件系统一致性检查
2  int fsck_verify(void) {
3      // 1. 超级块校验
4      if (!verify_superblock()) {
5          return -1;
6      }
7
8      // 2. inode完整性检查
9      for (uint inum = 1; inum < sb.ninodes; inum++) {
10         struct inode *ip = iget(dev, inum);
11         if (ip->type != 0) {
12             // 检查数据块有效性
13             if (!verify_inode_blocks(ip)) {
14                 printf("Inode %d has invalid blocks\n", inum);
15             }
16         }
17         iput(ip);
18     }
19
20     // 3. 位图与分配状态一致性
21     verify_block_allocation();
22
23     // 4. 目录结构完整性
24     verify_directory_structure();
25
26     return 0;

```

```

27 }
28
29 // 在线检查工具
30 void background_fsck(void) {
31     while (1) {
32         sleep(3600); // 每小时检查一次
33
34         if (filesystem_idle()) {
35             // 轻量级在线检查
36             online_consistency_check();
37         }
38     }
39 }

```

2. 自动修复策略:

- 孤儿 inode 处理: 链接到 lost+found 目录
- 交叉链接检测: 使用引用计数和反向指针
- 元数据损坏恢复: 从备份或日志中恢复

六、实验总结与改进方案

(一) xv6 文件系统优缺点总结

优点:

1. 教育价值: 结构清晰, 适合学习文件系统基本原理
2. 可靠性: 简单的设计减少了出错概率
3. 可理解性: 代码量适中, 便于深入分析

缺点:

1. 性能限制: 缺乏现代优化技术
2. 功能单一: 不支持高级特性如快照、压缩等
3. 扩展性差: 固定大小的设计限制了大规模应用

(二) 具体改进实施方案

阶段一: 性能优化

```

1 // 1. 实现读写分离的缓存策略
2 struct buffer_cache {
3     struct buffer_head *read_cache[READ_CACHE_SIZE];
4     struct buffer_head *write_cache[WRITE_CACHE_SIZE];
5     struct list_head clean_list;
6     struct list_head dirty_list;
7 };
8
9 // 2. 智能预读机制
10 void read_ahead(struct inode *ip, uint offset) {
11     uint next_block = offset / BSIZE + 1;
12     if (sequential_access_detected(ip)) {

```

```

13         // 预读后续块
14         for (int i = 0; i < READ_AHEAD_COUNT; i++) {
15             struct buffer_head *bh = get_block(ip->dev, next_block + i);
16             // 标记为预读, 低优先级
17             bh->priority = LOW_PRIORITY;
18             brelse(bh);
19         }
20     }
21 }

```

阶段二：功能扩展

```

1  // 1. 扩展属性支持
2  struct xattr_entry {
3      char name[XATTR_NAME_MAX];
4      uint size;
5      char value[XATTR_SIZE_MAX];
6  };
7
8  int setxattr(struct inode *ip, char *name, void *value, size_t size) {
9      // 小属性内联存储, 大属性使用外部块
10     if (size <= XATTR_INLINE_SIZE) {
11         // 存储在inode扩展区域
12         store_xattr_inline(ip, name, value, size);
13     } else {
14         // 分配专用数据块
15         store_xattr_external(ip, name, value, size);
16     }
17 }
18
19 // 2. 符号链接支持
20 int symlink(const char *target, const char *linkpath) {
21     struct inode *ip = create_file(linkpath, T_SYMLINK);
22     if (strlen(target) < 60) {
23         // 短链接内联存储
24         store_inline_symlink(ip, target);
25     } else {
26         // 长链接使用数据块
27         store_external_symlink(ip, target);
28     }
29     return 0;
30 }

```

阶段三：高级特性

```

1  // 1. 透明压缩
2  struct compression_info {
3      int algorithm;        // 压缩算法

```

```

4     int original_size; // 原始大小
5     int compressed_size; // 压缩后大小
6 };
7
8 int compressed_write(struct inode *ip, void *buf, uint size) {
9     void *compressed = compress_data(buf, size);
10    uint compressed_size = get_compressed_size();
11
12    if (compressed_size < size * COMPRESSION_THRESHOLD) {
13        // 存储压缩数据
14        write_compressed_blocks(ip, compressed, compressed_size);
15        set_compression_flag(ip);
16    } else {
17        // 存储原始数据
18        write_original_blocks(ip, buf, size);
19        clear_compression_flag(ip);
20    }
21 }
22
23 // 2. 数据校验和
24 struct checksum_block {
25     uint32_t checksum;
26     char data[BSIZE - 4];
27 };
28
29 int write_with_checksum(struct buffer_head *bh) {
30     struct checksum_block *cb = (struct checksum_block *)bh->data;
31     cb->checksum = calculate_checksum(cb->data, BSIZE - 4);
32     return bwrite(bh);
33 }

```

(三) 测试验证方案

性能基准测试

```

1 void comprehensive_benchmark(void) {
2     // 1. 元数据操作性能
3     benchmark_metadata_ops();
4
5     // 2. 文件I/O性能
6     benchmark_sequential_read();
7     benchmark_sequential_write();
8     benchmark_random_access();
9
10    // 3. 并发性能
11    benchmark_concurrent_access();
12
13    // 4. 恢复性能

```



```
14     benchmark_crash_recovery();
15 }
16
17 // 自动化回归测试
18 void regression_test_suite(void) {
19     struct test_case {
20         char *name;
21         void (*test_func)(void);
22         int expected_result;
23     };
24
25     struct test_case tests[] = {
26         {"create_delete", test_create_delete, 0},
27         {"large_file", test_large_file, 0},
28         {"concurrent_write", test_concurrent_write, 0},
29         {"power_failure", test_power_failure, 0},
30         // ... 更多测试用例
31     };
32
33     for (int i = 0; i < ARRAY_SIZE(tests); i++) {
34         printf("Running %s...", tests[i].name);
35         int result = tests[i].test_func();
36         if (result == tests[i].expected_result) {
37             printf("PASS\n");
38         } else {
39             printf("FAIL (expected %d, got %d)\n",
40                 tests[i].expected_result, result);
41         }
42     }
43 }
```

七、实验收获与展望

（一）核心技术收获

1. 文件系统架构理解：从磁盘布局到用户接口的完整栈理解
2. 一致性机制掌握：日志系统、事务处理、崩溃恢复等关键技术
3. 性能优化意识：缓存策略、数据结构选择、算法优化的重要性
4. 系统设计思维：在简单性、性能、功能之间的权衡决策

（二）未来研究方向

1. 分布式文件系统：将单机文件系统扩展为分布式架构
2. 新型存储介质：针对 NVMe、SCM 等新硬件的优化
3. AI 驱动优化：使用机器学习预测访问模式，优化缓存和预读
4. 安全增强：集成加密、访问控制、审计日志等安全特性

（三）工程实践建议

1. 渐进式开发：从简单可用的版本开始，逐步添加优化和功能
2. 测试驱动：为每个特性编写全面的测试用例
3. 性能分析：使用 `profiling` 工具识别真正的瓶颈
4. 文档维护：保持设计文档和代码注释的同步更新

通过本次文件系统实验，不仅深入理解了操作系统核心组件的工作原理，更重要的是培养了系统级软件的设计和实现能力，为后续更复杂系统的开发奠定了坚实基础。